The Omega Tower: Role Non-Impersonation and Authority Preservation in Agentic AI

Jeremy H. Carroll

2026-10-16

Version 1.3.0

Abstract

This monograph presents a contract model for separation of proposal, evaluation, and authorization in agentic systems. It verifies several algebraic components and records explicit failure modes when a declared contract is not honored.

We formalize the foundational boundary: **proposal is not authority** ($p \in \mathcal{P} \not\Rightarrow \mathsf{Auth}(p)$). Generating a fluent candidate action, regardless of heuristic confidence or internal self-critique, does not constitute execution authority. Authority is an inhabited dependent type requiring an explicit terminal certificate produced by an independent, replayable verification pipeline.

The six roles are a proposed contract: detect defects, gate authorization, track representation changes, compare candidates, check finite evidence, and retain unresolved outcomes. The mathematical results concern constructor distinctions, exception propagation, and predicate transport through a quotient. They do not prove that this six-role decomposition is necessary for every secure architecture.

---

1. Introduction: The Authority Problem in Agentic AI

Complex autonomous systems increasingly deploy large language models (LLMs) and heuristic planners within multi-agent loops to generate code, execute database mutations, call external APIs, and direct physical actuators. While individual model capabilities have expanded, the authority structures governing their deployment frequently remain informal.

A widespread failure pattern in agentic systems is architectural collapse:

- A component generating a proposal acts as its own evaluator or execution gate (self-authorization);
- An intermediate representation transformation (e.g., embedding, summarization, serialization) mutates semantic payload while discarding loss information (representation laundering);
- Multiple stochastic agents aggregate disparate outputs via majority vote or consensus, treating statistical plurality as formal correctness (consensus laundering);
- Unbounded continuous or long-horizon models claim general validity without testable, finite witnesses;
- Persistent defects or contradictions are silently erased or renamed as successful termination to avoid workflow halt (counterfeit closure).

In conventional software engineering, these collapses are often treated as ad-hoc bugs. However, in security-critical agentic execution, they represent systemic violations of separation of privilege (Saltzer & Schroeder, 1975) and commercial integrity (Clark & Wilson, 1987).

This paper introduces the **Omega Tower architecture**: a formal six-role framework designed to enforce role non-impersonation, fail-closed refusal propagation, and rigorous authority preservation.

---

2. Formal Framework: The Proposal/Authority Boundary

The architecture begins from a strict negative boundary:

$$\boxed{p \in \mathcal{P} \nRightarrow \mathsf{Auth}(p)}$$

Let $\mathcal{P}$ denote the set of candidate proposals (e.g., text, action plans, AST fragments, SQL queries). A proposal is an unverified input. Heuristic fluency is not verification; internal self-scoring is not independent evaluation; confidence is not certification.

2.1 The Dependent Authority Type

Authority is modeled as an inhabited dependent type:

$$\mathsf{Auth}(p) := \Sigma(t : \mathbb{T}_{\mathsf{auth}})\,(\Pi^{\sharp}(p) = \mathsf{Success}(t) \wedge \mathsf{IsValidToken}(p, t))$$

where:

- $\mathbb{T}_{\mathsf{auth}}$ is the type of terminal authority tokens;
- $\Pi^{\sharp} : \mathcal{P} \to \mathcal{M}(\mathbb{T})$ is the declared authority execution pipeline;
- $\mathcal{M}$ is the refusal monad $\mathcal{M}(X) = X + \mathbb{F}$, where $\mathbb{F}$ is the set of explicit refusal diagnostics;
- $\mathsf{IsValidToken}(p, t)$ is the token validity predicate.

Model Scope and Validity Predicate Differentiation:

1. **Lean Syntactic Validity Specification:** In the mechanized Lean 4 formalization (`AuthorityResidual.lean`), `isValidAuthorityToken` is defined strictly as a syntactic record check:

   $$\mathsf{isValidAuthorityToken}(p, t) := (t.\text{proposalId} = p.\text{id}) \wedge (t.\text{replayable} = \text{true}) \wedge (t.\text{verifier} \neq \texttt{""}) \wedge (t.\text{receiptDigest} \neq \texttt{""})$$

   Lean establishes structural theorems about this constructor specification: terminal disjointness ($\mathsf{Authority} \cap \mathsf{Residual} = \emptyset$, $\mathsf{Authority} \cap \mathsf{Refusal} = \emptyset$), left-zero absorption, and failure-mode non-factorization across untyped quotients. It checks syntactic well-formedness and field non-emptiness; it does not model dynamic stateful registry mutation or cryptographic signature issuance.
2. **Operational Registry Issuance:** In the executable Python implementation (`code/models.py`), authorization requires active validation by a verifier-owned receipt registry (`AuthorityReceiptRegistry.validates(p, t)`). This ensures the receipt ID was explicitly issued by the designated verifier for proposal p's canonical serialization digest. The proposal body (payload and action-relevant metadata) is hashed separately from the receipt/evidence envelope, ensuring attaching an issued token does not invalidate its own binding. Caller-asserted metadata flags (such as `verified_token=True`) are strictly rejected.
3. **Mathematical Classification vs. Authorization:** Evaluating whether a cochain defect vanishes ($r = 0$) or is exact ($r \in \operatorname{im} d_0$) is an algebraic cochain evaluation. Mathematical exactness is not institutional authorization. We enforce a strict separation of four distinct operations:

   $$\text{cochain evaluation} \longrightarrow \text{checked mathematical result} \longrightarrow \text{authorization decision} \longrightarrow \text{issued receipt}$$

   A mathematical classifier returns only defect classifications and supporting evidence; it does not manufacture authority tokens from unchecked raw numbers. An authority gate issues authority only after checking the applicable verification contract against that evidence and the proposal. Inconsistent numerical inputs (such as $r = 0$ with non-zero boundary) fail closed.

If the pipeline outputs a refusal $f \in \mathbb{F}$ or an unabsorbable residual r, the type $\mathsf{Auth}(p)$ is uninhabited. No action may execute without an explicit inhabitant of $\mathsf{Auth}(p)$.

2.2 Typing Boundaries and Scope Fences

We establish the following scope boundaries:

1. **Contract Authority vs. Semantic Truth:** Authority in this framework is strictly *Contract Authority* (relative to declared verification contracts) or *Verifier-Scoped Authority* (scoped to a named verifier's operational domain). We do not claim that verified authority ensures metaphysical truth; rather, it ensures under the declared assumptions that declared verification contracts were executed without refusal.
2. **Governance Positions, Not Ontological Levels:** The roles defined herein represent distinct functional obligations in a governance pipeline, not a hierarchy of being or intrinsic agent capabilities.

---

3. The Six Conserved Architectural Obligations

The architecture decomposes authority governance into six distinct functional roles:

$$\Omega_{\partial} \to \Omega_{\mathbb{A}} \to \Omega_{\chi} \to \Omega_{\mathfrak{D}} \to \Omega_{\kappa} \to \Omega_{\Lambda\Omega}$$

Table 1: The Six Conserved Governance Roles

Role	Name	Core Question	Governing Prohibition
Ω_{∂}	Obstruction Verifier	Is the defect zero, closed survivor, or open?	Cannot authorize without an explicit cochain evaluation receipt.
$\Omega_{\mathbb{A}}$	Finite Authority Gate	Does the candidate terminalize as authority or refusal?	Cannot authorize without independent verifier execution (D, V, τ).
Ω_{χ}	Certified Transport	Does representation change preserve authority?	Cannot create authority across representation shifts without loss accounting.
$\Omega_{\mathfrak{D}}$	Comparison Envelope	Can candidates be compared without laundering?	Cannot mint execution authority via statistical voting or consensus aggregation.
Ω_{κ}	Finite-Shadow Projection	Do finite shadows cohere, and is the query verified?	Cannot create authority from unbounded continuum coherence alone.
$\Omega_{\Lambda\Omega}$	Typed Residual Closure	What remains when closure cannot absorb the residual?	Cannot create authority by erasing or renaming unabsorbable conflict.

Each role names an obligation of this contract. Compression of components in an implementation is admissible only if the resulting system continues to discharge all six obligations and preserves the corresponding failure terminals.

---

4. The Collapse Space: Modes, Counterexamples, and Invertibility

The contract associates omitted obligations with explicit failure labels when authorization is attempted.

The implementation assigns one distinct failure constructor to each of the six role constructors. This correspondence is a classification convention; real failures may have several causes. A constructor bijection is not a diagnosis of an arbitrary deployed system.

Table 2: Declared constructor correspondence

Role constructor	Failure constructor
obstruction	undetectedObstruction
authority	proposalBecomesAction
transport	silentRepresentationMutation
comparison	comparisonBecomesLaundering
shadow	continuumShadowBecomesFakeAuthority
residual	residualGetsHiddenOrRenamed

Theorem 1 (Constructor Invertibility and Role Distinction)

Let $\mathcal{R} = \{\Omega_\partial, \Omega_\mathbb{A}, \Omega_\chi, \Omega_\mathfrak{O}, \Omega_\kappa, \Omega_{\Lambda\Omega}\}$ and let $\mathcal{F}$ be the set of six characteristic failure modes in Table 2. The mapping $\phi : \mathcal{R} \to \mathcal{F}$ is bijective. Consequently:

1. *Injectivity:* $\forall r_1, r_2 \in \mathcal{R},\ r_1 \neq r_2 \implies \phi(r_1) \neq \phi(r_2)$.
2. *Invertibility:* There exists a unique left- and right-inverse $\phi^{-1} : \mathcal{F} \to \mathcal{R}$ such that $\phi^{-1}(\phi(r)) = r$ and $\phi(\phi^{-1}(f)) = f$.

Proof. Verified by direct construction and exhaustion in Lean 4 (`RoleNonImpersonation.RoleFailure`). The inverse table maps each failure label back to its assigned role. □

---

5. Worked Case Study: Configuration Conflict over $\mathbb{F}_2^3$

To demonstrate how role collapse induces detectable failure, we examine a concrete configuration reconciliation task over the vector space $\mathbb{F}_2^3$.

5.1 Scenario Setup

Consider three configuration keys:

1. `key_1`: Network Timeout Policy,
2. `key_2`: Security Handshake Protocol,
3. `key_3`: Logging Telemetry Level.

Let two independent sources A and B propose configuration states $c_A, c_B \in \mathbb{F}_2^3$:

- $c_A = (1, 0, 0)$
- $c_B = (1, 1, 0)$

The difference cochain $r \in \mathbb{F}_2^3$ evaluates coordinate-wise XOR over $\mathbb{F}_2$:

$$r = c_A \oplus c_B = (1 \oplus 1,\ 0 \oplus 1,\ 0 \oplus 0) = (0, 1, 0)$$

Sources A and B strictly disagree on `key_2`.

Use the declared complex $C^0 = \mathbb{F}_2 \xrightarrow{d_0} C^1 = \mathbb{F}_2^3 \xrightarrow{d_1} C^2 = \mathbb{F}_2$, where $d_0(a) = (a, 0, 0)$ and $d_1(x, y, z) = z$. Thus $d_1(r) = 0$, while $r \notin \mathrm{im}(d_0)$, so this example is closed and non-exact in the stated complex.

Thus, r represents a **closed, non-exact residual**:

$$r \neq (0, 0, 0) \quad \text{and} \quad \bar{d_1}(r) = 0$$

5.2 Compliant Full-Stack Execution

Under a compliant architecture with active Residual Closure ($\Omega_{\Lambda\Omega}$):

1. Ω_∂ computes $r = (0, 1, 0) \neq 0$ and $\bar{d}_1(r) = 0$;
2. $\Omega_{\Lambda\Omega}$ recognizes that no certified coboundary repair α exists such that $d_0\alpha = r$;
3. The system halts execution and emits the typed terminal TerminalStatus.RESIDUAL_CLOSED ("Unresolved conflict on key 2; human governance decision required").
4. The system **houses the conflict honestly** rather than fabricating consensus.

5.3 Collapsed Execution (Removal of $\Omega_{\Lambda\Omega}$)

Suppose an engineer removes the residual closure role $\Omega_{\Lambda\Omega}$ to prevent workflow interruptions.

1. The pipeline lacks a typed residual terminal;
2. The comparison role $\Omega_{\mathfrak{D}}$ attempts to reconcile the outputs without a residual home;
3. To avoid failing, the system forcibly outputs TerminalStatus.AUTHORITY marked as ResolvedExact;
4. An external auditor replaying the cochain computation finds $r = (0, 1, 0) \neq 0$ with no certified repair witness.
5. The auditor immediately flags ResidualGetsHiddenOrRenamed. The collapse is detected directly from the audit ledger row alone.

5.4 Maximal Sound Authority Descent Through a Quotient

For an authority predicate $A : X \to \text{Prop}$ and a representation transport map $q : X \to Y$, information loss across fibers $q^{-1}(y)$ creates a fundamental authority preservation problem. Following Lawvere (1969, 1970), universal quantification along a map is the right adjoint to inverse image (pullback):

$$\forall_q A(y) := \forall x \in X,\, q(x) = y \implies A(x)$$

We write $\forall_q A$ for this maximal sound descent predicate A_{safe}. Identifying predicates with their satisfying subsets, the central adjunction theorem is:

$$\boxed{q^{-1}(B) \subseteq A \iff B \subseteq \forall_q A}$$

Proof. Suppose $q^{-1}(B) \subseteq A$. If $y \in B$ and $q(x) = y$, then $x \in q^{-1}(B)$, which implies $x \in A$. Hence $y \in \forall_q A$, establishing $B \subseteq \forall_q A$.
Conversely, suppose $B \subseteq \forall_q A$. If $x \in q^{-1}(B)$, then $q(x) \in B \subseteq \forall_q A$. Applying $(\forall_q A)(q(x))$ to x with $q(x) = q(x)$ yields $x \in A$. Hence $q^{-1}(B) \subseteq A$. □

The adjunction has the following consequences:

1. **Soundness:** Pulling back the descended predicate $\forall_q A$ along q satisfies $q^{-1}(\forall_q A) \subseteq A$, i.e., $(\forall_q A)(q(x)) \implies A(x)$ for all $x \in X$.
2. **Maximality:** If $B : Y \to \text{Prop}$ is any sound authorization predicate on Y (meaning $q^{-1}(B) \subseteq A$), then $B \subseteq \forall_q A$. Thus $\forall_q A$ is the maximal sound descent predicate.
3. **Exact Preservation on Fiber-Constant Predicates:** Lossless authority descent $(\forall_q A)(q(x)) \iff A(x)$ holds for all $x \in X$ if and only if A is constant on the fibers of q:
$$\forall x, x' \in X,\, q(x) = q(x') \implies (A(x) \iff A(x'))$$
4. **Mixed Fiber Obstruction:** If a fiber contains both an authorized state x_+ and an unauthorized state x_-:
$$q(x_+) = q(x_-), \qquad A(x_+), \qquad \neg A(x_-)$$
then no predicate B on Y can simultaneously reproduce both authority judgments: $(B(q(x_+)) \iff A(x_+)) \wedge (B(q(x_-)) \iff A(x_-))$ is a provable contradiction.

5. **Compositionality:** For successive transports $q_1 : X \to Y$ and $q_2 : Y \to Z$, universal quantification composes identically to map composition:

$$\forall_{q_2}(\forall_{q_1} A) = \forall_{q_2 \circ q_1} A$$

6. **Distribution over Conjunction of Obligations:** For a family of obligation predicates $A_i : X \to \mathrm{Prop}$, universal quantification distributes across finite conjunctions:

$$\boxed{\forall_q \left(\bigwedge_i A_i \right) = \bigwedge_i \forall_q A_i}$$

Proof. Follows immediately by interchanging universal quantifiers: $\forall x, (q(x) = y \implies \bigwedge_i A_i(x)) \iff \bigwedge_i (\forall x, q(x) = y \implies A_i(x))$. □

Empty fibers. If $y \notin q(X)$, the predicate $(\forall_q A)(y)$ is vacuously true. An operational gate must restrict its domain to $q(X)$ or require a source witness in addition to this predicate. Universal quantification alone does not authorize a fabricated representation with no preimage.

This provides a domain-agnostic mathematical account of transporting a conjunction of obligations. The six named governance roles $\bigwedge_{i=1}^{6} \Omega_i$ constitute a concrete application instance of this theorem, while the role-erasure quotient boundary (`QuotientBoundary.lean`) is a specialization of the mixed fiber obstruction. All seven descent theorems are machine-checked in Lean 4 (`RoleNonImpersonation.AuthorityDescent`) with zero custom axioms.

---

6. Formal Non-Impersonation and the Kleisli Pipeline Theorem

We formalize the execution pipeline using monads and Kleisli categories.

6.1 The Refusal Monad

Let $\mathcal{M}(X) = X + \mathbb{F}$ denote the refusal monad (an instance of the exception monad), where X represents valid output and $\mathbb{F}$ represents explicit, typed refusal diagnostics.

The monad operations are:

$$\mathsf{pure}(x) = \mathsf{Success}(x)$$

$$\mathsf{bind}(m, f) = \begin{cases} f(x) & \text{if } m = \mathsf{Success}(x) \\ \mathsf{Refusal}(err) & \text{if } m = \mathsf{Refusal}(err) \end{cases}$$

Proposition 1 (Left-Zero Absorption)

For any refusal diagnostic $err \in \mathbb{F}$ and any continuation morphism $f : X \to \mathcal{M}(Y)$:

$$\mathsf{bind}(\mathsf{Refusal}(err), f) = \mathsf{Refusal}(err)$$

Proof. Immediate from the definition of bind. Verified in Lean 4 (`RoleNonImpersonation.ExceptionPipeline.bindR_refusal_left_zero`). □

6.2 The Five-Stage Kleisli Factorization

The declared five-stage pipeline is defined as a composite morphism in the Kleisli category of $\mathcal{M}$:

$$\Pi^\sharp = \tau^\sharp \circ_K V^\sharp \circ_K D^\sharp \circ_K \rho^\sharp \circ_K \mathcal{R}^\sharp$$

where:

- $\mathcal{R}^\sharp : \mathcal{P} \to \mathcal{M}(\mathcal{P}_1)$ evaluates defect cochains (Role Ω_∂);
- $\rho^\sharp : \mathcal{P}_1 \to \mathcal{M}(\mathcal{P}_2)$ validates representation transport and loss ledgers (Role Ω_χ);
- $D^\sharp : \mathcal{P}_2 \to \mathcal{M}(\mathcal{P}_3)$ computes descent candidates (Role $\Omega_\mathbb{A}$ preparation);
- $V^\sharp : \mathcal{P}_3 \to \mathcal{M}(\mathcal{P}_4)$ executes independent verification (Role $\Omega_\mathbb{A}$ check);
- $\tau^\sharp : \mathcal{P}_4 \to \mathcal{M}(\mathbb{T})$ terminalizes the execution into an authority token or typed residual (Roles $\Omega_\mathbb{A}$ and $\Omega_{\Lambda\Omega}$).

The comparison obligation ($\Omega_\mathfrak{D}$) and finite-shadow obligation (Ω_κ) act as necessary side-condition guards entering $V^\sharp$ and $\tau^\sharp$.

Theorem 2 (Refusal Propagation and Anti-Cheat)

If any stage S_i in the Kleisli factorization returns a refusal $\mathsf{Refusal}(err)$*, the complete pipeline evaluates to that exact refusal:*

$$\Pi^\sharp(p) = \mathsf{Refusal}(err)$$

No subsequent stage $S_{j>i}$ is executed, and no authority token is produced.

Proof. Follows by induction on the pipeline length using Proposition 1 (associativity and left-zero absorption). Machine-checked in Lean 4 (`RoleNonImpersonation.ExceptionPipeline.pipeline5_*_refusal_propagates`). □

Operational Premise on Side-Effect Absorption: Under pure refusal monad absorption, subsequent callbacks are left uncalled and the composite pipeline evaluates to $\mathsf{Refusal}(err)$. However, short-circuiting does not automatically rollback side effects executed prior to refusal. The operational invariant that "no authority token is produced" requires that preterminal stages ($S_{i<5}$) are strictly unprivileged evaluators lacking registry issuance capabilities, with token creation isolated exclusively to the designated terminal authority gate $\tau^\sharp$. In operational architectures, one must distinguish:

$$\text{refusal propagation} \neq \text{rollback} \neq \text{control of issuance capabilities.}$$

Contract Rule 3 (The Role Non-Impersonation Principle)

Within the declared contract, an undischarged required obligation prevents successful authorization unless an accepted equivalent discharges that obligation. Under this closed routing policy, removing a role without an approved replacement routes to its mapped failure label upon attempted authorization. This is a contractual routing policy, not an architectural-necessity theorem: an architecture that unconditionally refuses all proposals omits the roles without executing unauthorized actions.

Contract Interpretation and Illustrative Omission Scenarios. Under this declared contract:

- If Obstruction Detection (Ω_∂) is omitted, candidate defects are uncomputed; any attempt to grant authority routes to UndetectedObstruction.
- If Authority Gating ($\Omega_\mathbb{A}$) is omitted, unverified proposals pass directly to execution, routing to ProposalBecomesAction.
- If Certified Transport (Ω_χ) is omitted, representation transformations lack loss ledgers, routing to SilentRepresentationMutation.
- If Comparison Envelope ($\Omega_\mathfrak{D}$) is omitted, conflicting outputs are averaged or voted without typed comparison, routing to ComparisonBecomesLaundering.
- If Finite-Shadow Projection (Ω_κ) is omitted, continuum models claim unbounded authority without query-relative finite projections, routing to ContinuumShadowBecomesFakeAuthority.
- If Typed Residual Closure ($\Omega_{\Lambda\Omega}$) is omitted, unabsorbable conflicts lack a lawful non-execution terminal, routing to ResidualGetsHiddenOrRenamed.

The mechanized Lean formalization verifies constructor bijections, injectivity, refusal absorption, and quotient boundaries; it formalizes the declared contractual routing, not an architectural-necessity claim.

---

7. Query-Relative Finite Shadows for Continuous Models

In machine learning and agentic workflows, many models operate over continuous spaces (e.g., real-valued neural network weights, high-dimensional latent embeddings, probability distributions, continuous time trajectories).

We assert the finite-shadow principle:

Continuum coherence is not authority. Operational authority is finite-shadow, query-relative, and certificate-compatible.

This does not imply that continuous mathematics is invalid. Rather, it recognizes that in physical and computational systems, an execution decision is discrete and must be verified against finite, inspectable evidence.

7.1 Finite coverage and query factorization

A finite set of checked projections establishes only the declared finite coverage contract. For an exact query $Q : X \to T$, a projection $q_J : X \to Y_J$ suffices precisely when there is a function $\widehat{Q} : q_J(X) \to T$ with

$$Q = \widehat{Q} \circ q_J.$$

Equivalently, Q must be constant on the fibers of q_J. This is a specification to justify for each query. The Python finite-shadow checker checks supplied coherence data; it does not prove this factorization or grant authorization. A finite collection of projections is not automatically cofinal in an infinite index category; for example, no finite subset is cofinal in $(\mathbb{N}, \leq)$.

---

8. Typed Residual Closure and Honest Refusal ("I Do Not Know")

A primary driver of catastrophic agentic failure is the refusal to fail honestly. When an agent cannot solve a task or reconcile conflicting requirements, conventional systems often hallucinate an answer, average the contradiction, or loop indefinitely.

The Residual Closure role ($\Omega_{\Lambda\Omega}$) provides a typed, lawful terminal for unabsorbable conflict.

Table 3: Mathematical classes and operational outcomes

Mathematically, let $\text{Exact}(r) \iff \exists \alpha,\ d_0\alpha = r$ and $\text{Closed}(r) \iff d_1 r = 0$ in the declared complex. We distinguish:

1. $r = 0$: Zero defect;
2. $r \neq 0$ and $\text{Exact}(r)$: Nonzero but repairable defect;
3. $\text{Closed}(r)$ and $\neg\,\text{Exact}(r)$: Closed, non-exact residual;
4. $\neg\,\text{Closed}(r)$: Open residual.

Mathematical Condition	Terminal Constructor	Operational Interpretation	Action Implication
Zero defect $r = 0$	ResolvedExact	Exact closure evaluated; invariant holds.	Requires distinct verification gate to issue authority.
Nonzero exact defect $r \neq 0$, $\text{Exact}(r)$	ResolvedExact	Nonzero repairable defect in $\text{im}(d_0)$.	Requires verified repair witness α; authority withheld otherwise.

Mathematical Condition	Terminal Constructor	Operational Interpretation	Action Implication
Closed non-exact residual $\bar{d}_1(r) = 0, \neg\,\mathrm{Exact}(r)$	ClosedResidual	Stable survivor defect; real unabsorbable conflict.	Block execution; route to human governance.
Open boundary defect $\bar{d}_1(r) \neq 0$	OpenResidual	Open boundary condition; incomplete evaluation.	Block execution; request additional evidence.
Verified through dimension N	CoherentUpTo(N)	Finite-horizon honesty; bounded coherence.	Coherence within horizon; does not grant execution authority.
Computational budget exhausted	BudgetUnknown	Resource ceiling reached; honest "I do not know".	Halt immediately; refuse speculative completion.

The BudgetUnknown constructor records an incomplete computation. The label alone does not certify that the budget was enforced; that requires an execution record.

---

9. Replay Custody, Verification Receipts, and Operator Memory

The Python reference model uses a verifier-owned in-memory receipt registry. It prevents a caller-supplied token from passing the local contract check, but it is not cryptographic custody, persistence, or independent third-party verification.

9.1 Verification Receipt Structure

The following is an illustrative schema, not a cryptographic protocol or a claim of independent verification:

```
{
  "proposal_id": 10482,
  "proposal_hash": "sha256:7f83b165...",
  "verifier_identity": "cochain_exactness_oracle_v2",
  "replayable": true,
  "execution_receipt": {
    "timestamp_epoch": 1788825600,
    "stages_passed": ["obstruction", "transport", "descent", "verify", "terminalize"],
    "defect_norm": "0.0",
    "receipt_digest": "sha256:e3b0c442..."
  }
}
```

In the reference model, replay means that the issuing in-memory registry recognizes the exact proposal binding. It provides neither cross-process custody nor cryptographic authenticity.

9.2 Reuse after a new check

A stored successful run may inform a later proposal. Reusing it requires checking the current proposal, context, contract, and evidence again. Persistent, authenticated receipts and independent custody are future implementation work; the shipped in-memory registry does not supply them.

---

10. Deterministic Adversarial Fixture Suite

This deterministic fixture suite evaluates 20 hand-authored strings and metadata mappings against fixed rules (`code/adversarial_benchmark.py`). It performs no LLM invocation, sampling, or empirical model comparison.

The suite includes 16 adversarially labelled fixtures and four benign fixtures covering input injection strings, role flags, payload mutation, capability checks, confidence claims, and receipt replay. These are hand-authored regression cases, not a sampled threat population.

10.2 Fixture Classification Results

The baseline is a deliberately hard-coded always-allow classifier. Its results are a reference row, not a measured baseline agent.

Table 4: Deterministic Fixture Classification

Metric	Always-allow classifier	Five-Stage Authority Pipeline	Architectural Meaning
Total Test Cases	20	20	Complete benchmark suite evaluated
Adversarial Attack Cases	16	16	Prompt injection, tool hijacking, role tampering, and spoofing
Attacks Intercepted	0 / 16 (0.0%)	16 / 16 (100.0%)	Fail-closed refusal on all adversarial attempts
Benign Workflows Preserved	4 / 4 (100.0%)	4 / 4 (100.0%)	Legitimate proposals reach valid terminal authority
False Refusals (False Positives)	0 / 4 (0.0%)	0 / 4 (0.0%)	No benign fixture rejected
Overall Accuracy	4 / 20 (20.0%)	20 / 20 (100.0%)	Correct classification and routing across all inputs
Attack Detection Recall	0.0%	100.0%	All listed adversarial fixtures rejected
Benign Specificity	100.0%	100.0%	Clean preservation of valid capability grants

10.3 Fixture-Suite Claim Boundaries

Under the declared Claim Hygiene protocol:

- **What this supports:** The fixed rules classify the 20 checked fixtures as expected.
- **What this does not support:** It is not an LLM evaluation, a security benchmark, or evidence of cryptographic or independent verification.
- **Narrowest defensible claim:** The reference contract refuses the listed malformed fixture inputs and registry validation rejects a hand-written receipt.

---

11. Scope, Limitations, and Falsifiability

In accordance with strict claim hygiene, we make the following explicit non-claims:

1. **No Claim of Unconstrained AI Safety:** This monograph does not claim that the Omega Tower makes AI "unconditionally safe" or solves alignment in an absolute sense. It provides an architectural mechanism for fail-closed separation of privilege and role accountability.
2. **Compression Admissibility:** We do not claim that six separate software microservices must exist in every implementation. Components may be fused provided all six functional obligations and typed failure terminals are preserved.
3. **Implementation Falsifiability Challenge:** The architecture is falsifiable. If an investigator demonstrates an accepted terminal authority outcome for which any required obligation lacks valid supporting evidence under the declared contract, the authority gate implementation is refuted.

---

12. References

- Clark, D. D., & Wilson, D. R. (1987). A comparison of commercial and military computer security policies. In *IEEE Symposium on Security and Privacy* (pp. 184-195).
- Dennis, J. B., & Van Horn, E. C. (1966). Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3), 143-155.
- Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H., & Winwood, S. (2009). seL4: Formal verification of an OS kernel. In *ACM SIGOPS Operating Systems Review* (Vol. 43, No. 4, pp. 207-220).
- Lawvere, F. W. (1969). Adjointness in foundations. *Dialectica*, 23(3/4), 281-296.
- Lawvere, F. W. (1970). Equality in hyperdoctrines and comprehension schema as an adjoint functor. In *Applications of Categorical Algebra* (Proc. Sympos. Pure Math., Vol. XVII, pp. 1-14). AMS.
- Miller, M. S., Tulloh, B., & Shapiro, J. S. (2005). The Structure of Authority: Why Security Is Not a Separable Concern. In *Multiparadigm Programming in Mozart/Oz* (Lecture Notes in Computer Science, Vol. 3389, pp. 2-20). Springer.
- Moggi, E. (1991). Notions of computation and monads. *Information and Computation*, 93(1), 55-92.
- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278-1308.
- Wadler, P. (1992). The essence of functional programming. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (pp. 1-14).
- Zheng, L., Chiang, W. L., Sheng, Y., Tian, S., Wu, C., Zhuang, H., Hao, Z., Zhu, B., Kwon, W., Liu, D., & Xing, E. P. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36.

Appendix A: Summary of Machine-Checked Lean 4 Kernels

The formal results in this monograph are verified in a self-contained Lean 4 package (`lean4/`), compiling under toolchain `v4.34.0` with zero `sorry` placeholders and strictly audited standard axioms (`Quot.sound`, `propext`):

1. `RoleFailure.lean`:
   - Enumeration of `Role` (6 constructors) and `FailureMode` (6 constructors).
   - Bijective mappings `roleToFailure` and `failureToRole`.
   - Proofs: `failureToRole_roleToFailure`, `roleToFailure_failureToRole`, `roleToFailure_injective`, `roleToFailure_ne_of_ne`, `failureToRole_injective`.
   - Formalization of `collapseOutcome` and `collapse_failure_identifies_removed_role`.
   - Obligation vector readiness: `missing_obligation_not_ready`.
2. `ExceptionPipeline.lean`:
   - Refusal monad `RefusalM T = success T | refusal String`.
   - Kleisli bind and Monad laws (`bind_pure`, `bind_right_id`, `bindR_assoc`).
   - Left-zero absorption: `bindR_refusal_left_zero`.
   - Five-stage pipeline `pipeline5` and refusal propagation theorems for all five stages.
3. `AuthorityResidual.lean`:
   - Proposal structure and `AuthorityToken` with replay custody.
   - Terminal constructors: `authority`, `residualClosed`, `residualOpen`, `refusal`, `budgetUnknown`.
   - Terminal constructor disjointness: `authority_not_residual`, `authority_not_refusal`, `residual_not_refusal`.
   - Formal dependent type `Auth(p)` requiring valid, replayable tokens.
   - Non-authority theorems for residual, refusal, and unreplayable tokens.
   - Cohomological defect classification and the `non_exact_residual_not_resolved` and `renamed_non_exact_as_resolved_contradiction` theorems.
4. `QuotientBoundary.lean`:
   - Formalization of typed packets carrying `(payload, role)`.
   - Definition of payload-erasure quotient using `Setoid` and `Quotient`.
   - Two-way bridge theorem `payload_classifier_factors_iff`.

- Direct non-existence theorem `role_failure_no_quotient_factor`: No quotient factor exists on the payload-only quotient for the role failure classifier.
- Theorem `role_failure_not_factor_through_payload_quotient`: Fiber invariance fails across distinct roles.
- Theorem `payload_green_role_failure_red`: Equal payloads may carry different role-failure labels; the role-sensitive classifier therefore cannot be recovered from the payload-only quotient.

5. `AuthorityDescent.lean`:
   - Formalization of Lawvere universal quantification $\forall_q A(y) := \forall x, q(x) = y \to A(x)$.
   - Adjunction theorem `lawvere_adjunction`: $q^{-1}(B) \subseteq A \iff B \subseteq \forall_q A$.
   - Theorem `descent_soundness`: Sound authority descent along any projection.
   - Theorem `descent_maximality`: Any sound transported authority predicate factors through $\forall_q A$.
   - Theorem `exact_preservation_iff_fiber_constant`: Exact descent holds if and only if authority is constant on fibers.
   - Theorem `mixed_fiber_obstruction`: A mixed fiber prevents lossless authority transport.
   - Theorem `descent_composition`: Adjoint functoriality under projection composition $\forall_{q_2}(\forall_{q_1} A) = \forall_{q_2 \circ q_1} A$.
   - Theorem `descent_conjunction`: Distribution of universal quantification across obligation conjunctions.
